International Cybersecurity and Digital Forensics Academy (ICDFA)

Digital Forensic Investigation Practical

Steganography: LSB Substitution, Steghide Embedding/Extraction and StegSeek Password Recovery

Digital Forensic Report

Field	Detail
Student Name	Mary-Joy Adewole
Registration Number / Student ID	2025/FWSD/11468
Course Code	SBT-DF202
Course Title	Computer and Digital Forensics
Practical Title	Steganography and Hidden Data Analysis
Evidence Items	_tower_original_image_for_lab.bmp (carrier); two independently created secret files; two Steghide stego images; two custom wordlists
Date of Analysis	4 September 2026
Date of Submission	4/09/2026
Instructor Name	Aminu Idris, AMCPN

1. Procedure

This practical applied core steganography and steganalysis techniques using a single BMP carrier image: embedding a secret text file using Steghide, verifying successful embedding and extraction through cryptographic hashing and byte-level comparison, and recovering the embedding passphrase using StegSeek against a custom wordlist. The exercise was performed twice - once following the guided lab scenario, and once independently with a self-authored secret file, passphrase, and wordlist - to demonstrate the technique end-to-end rather than merely reproducing a single worked example. Module 8 study material on steganography, LSB substitution and bitmap structures, and password recovery with wordlists was reviewed and completed in full before any command was executed.

1.1 Module 8 Pre-Lab Review and Environment Preparation

Completion of the Module 8 coursework (Steganography and Hidden Data Analysis) was confirmed via the ICDFA E-Campus portal before starting, alongside the preceding Data Carving and File Recovery Techniques module.

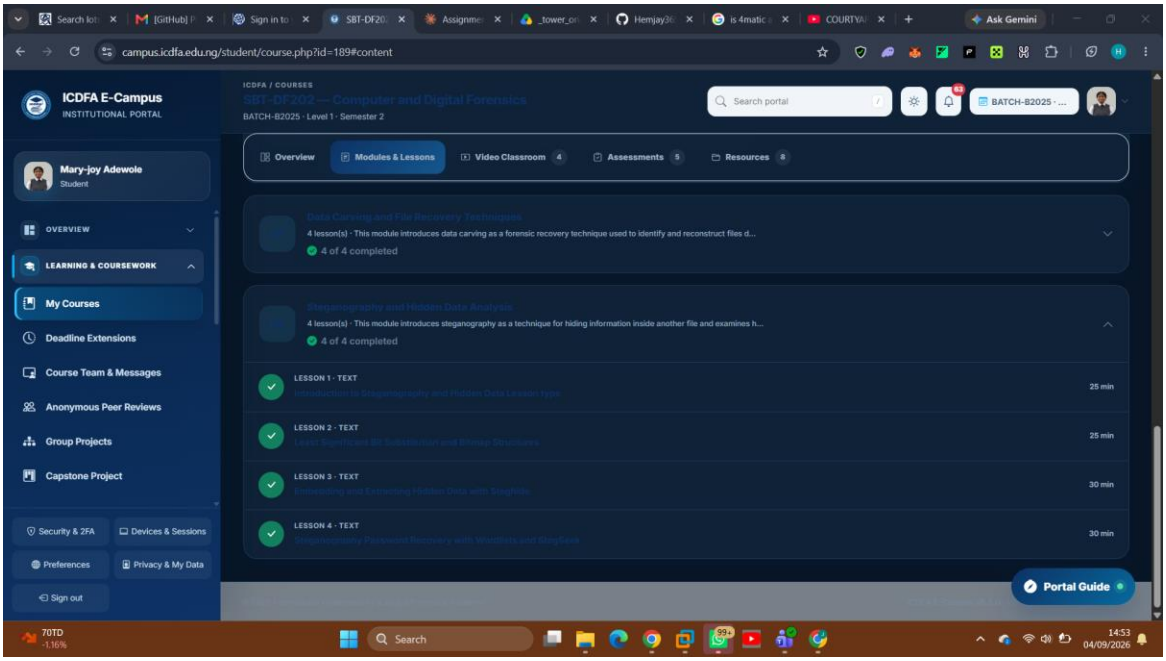


Figure 1: ICDFA E-Campus portal confirming all 4 lessons of "Steganography and Hidden Data Analysis" (and the preceding module) completed prior to starting the practical.

The required tools, Steghide and StegSeek, were confirmed present, with StegSeek upgraded to ensure the latest version was in use.

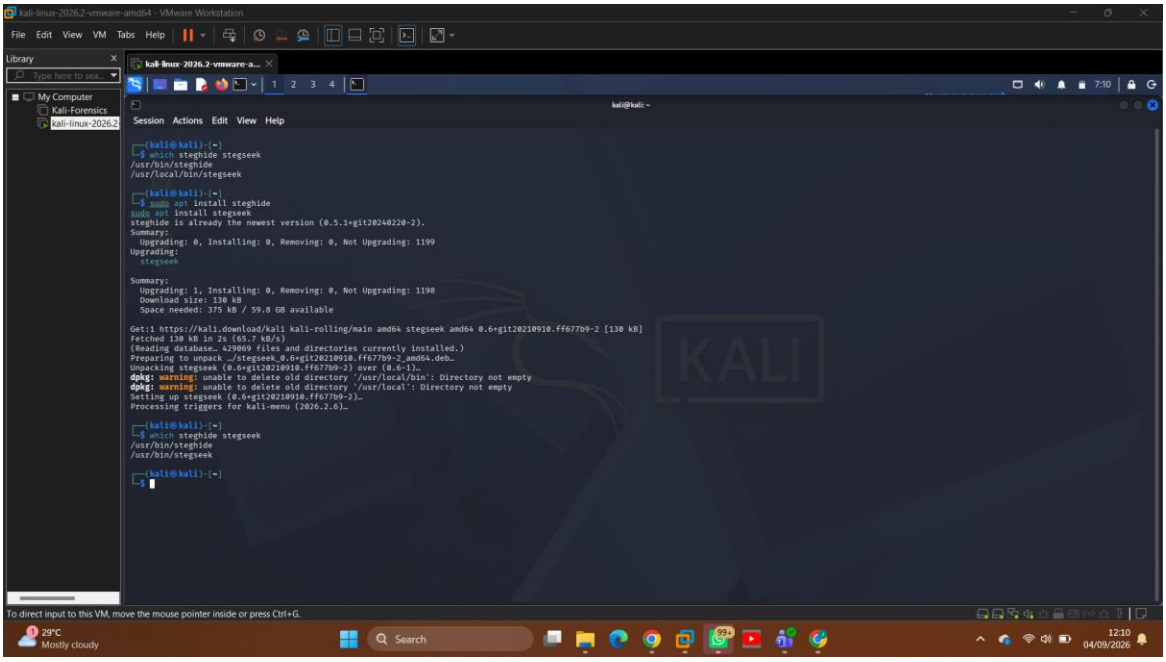


Figure 2: Confirming steghide and stegseek are installed via which, then upgrading stegseek to the latest available version.

2. Part A - Carrier Image Preparation

The carrier image was transferred from Windows via the VMware shared folder and copied into a dedicated carrier/ directory.

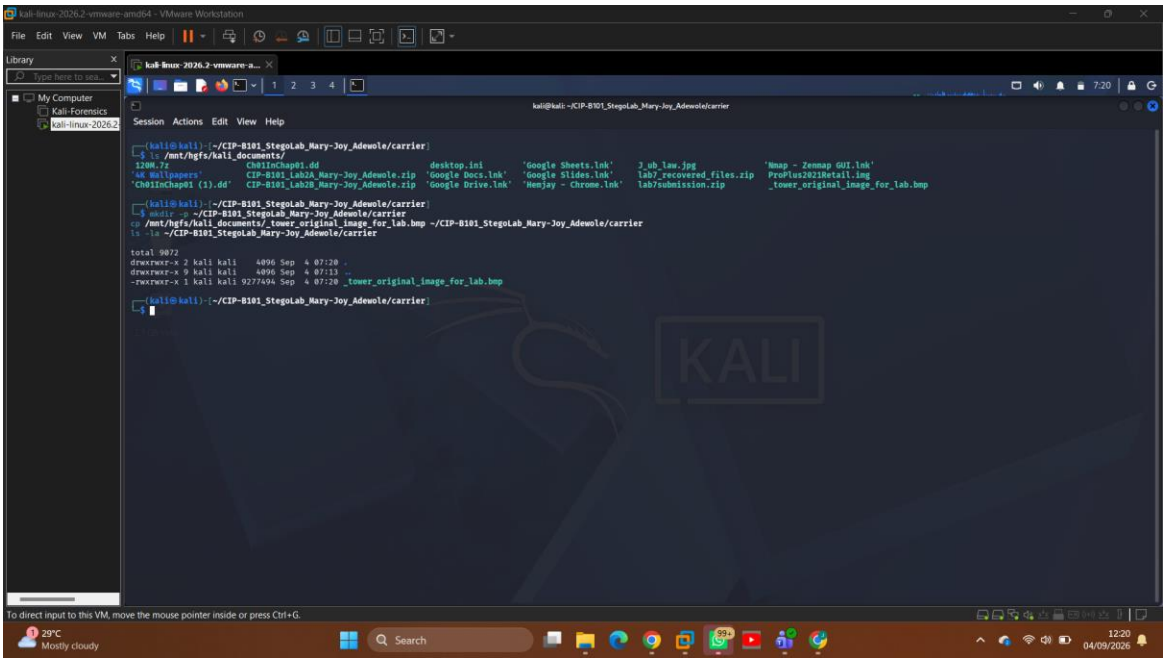


Figure 3: Carrier image (_tower_original_image_for_lab.bmp) copied from the shared folder into the working directory structure.

The carrier's file type and structural details were confirmed with file, and a baseline SHA-256 hash was computed before any embedding took place.

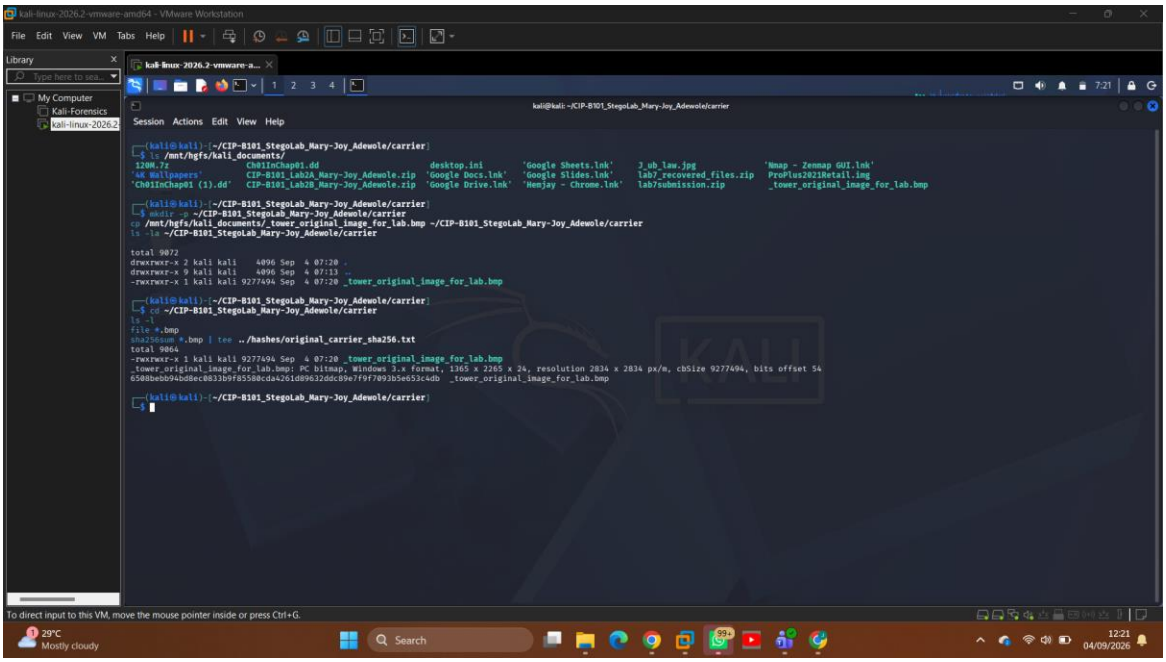


Figure 4: file confirming BMP structure (1365x2265x24, Windows 3.x format) and sha256sum recording the baseline carrier hash.

Carrier Image Details:

Filename	_tower_original_image_for_lab.bmp
Format	PC bitmap, Windows 3.x format
Dimensions	1365 x 2265 x 24
Resolution	2834 x 2834 px/m
File size	9,277,494 bytes
SHA-256 (original)	6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db

3. Part B - Secret File Creation (Guided Exercise)

A secret text file was created containing identifying and confirmation information for this lab exercise, then hashed immediately.

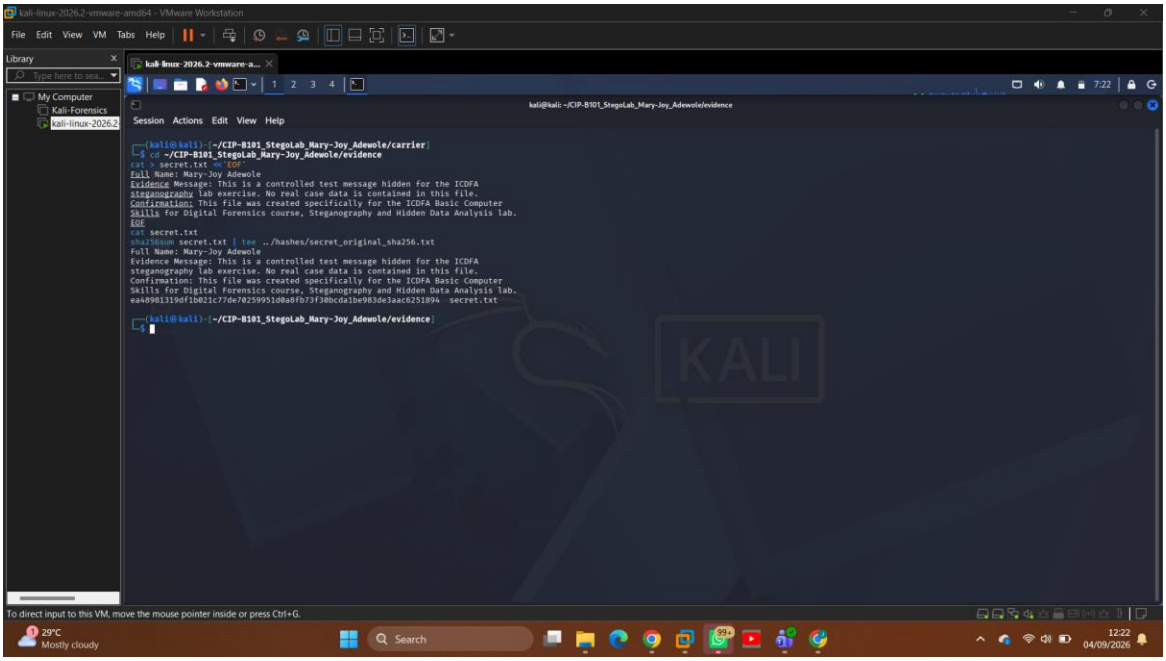


Figure 5: evidence/secret.txt created with student name, evidence statement and course confirmation; content displayed via cat; SHA-256 hash recorded.

4. Part C - Steghide Embedding

The secret file was embedded into the carrier image using Steghide with a defined passphrase, producing the stego image.

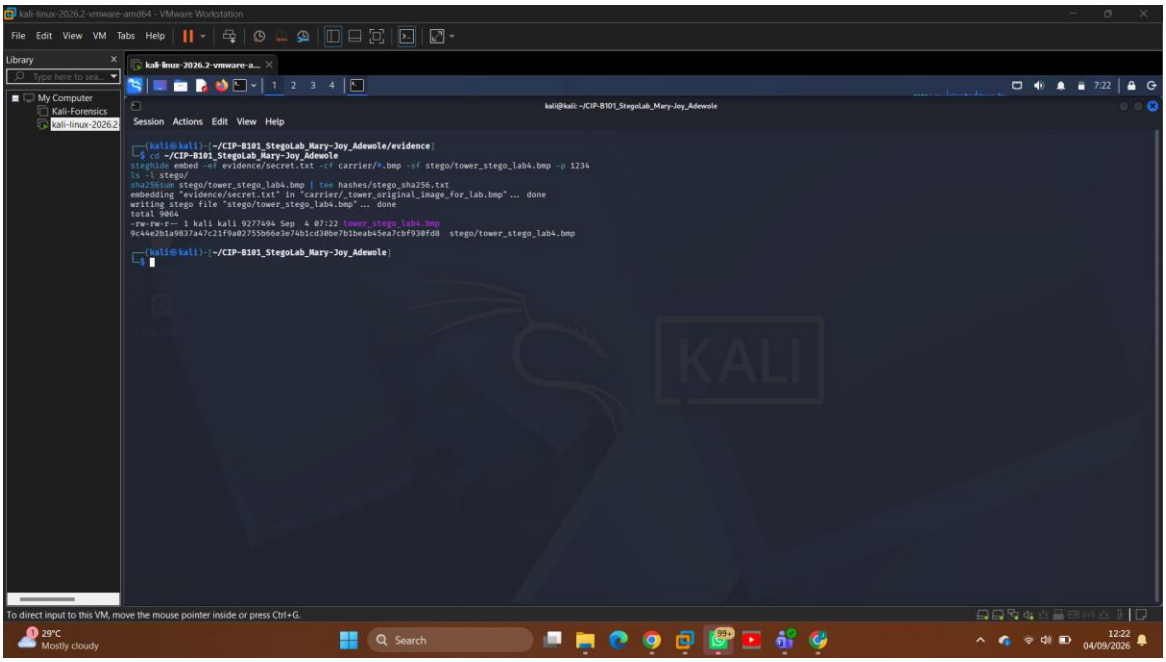


Figure 6: steghide embed command with passphrase "1234"; resulting stego image listed and hashed.

5. Part D - Extraction and Verification

The embedded content was extracted back out using the same passphrase, then compared byte-for-byte against the original secret file using diff, alongside an independent hash comparison.

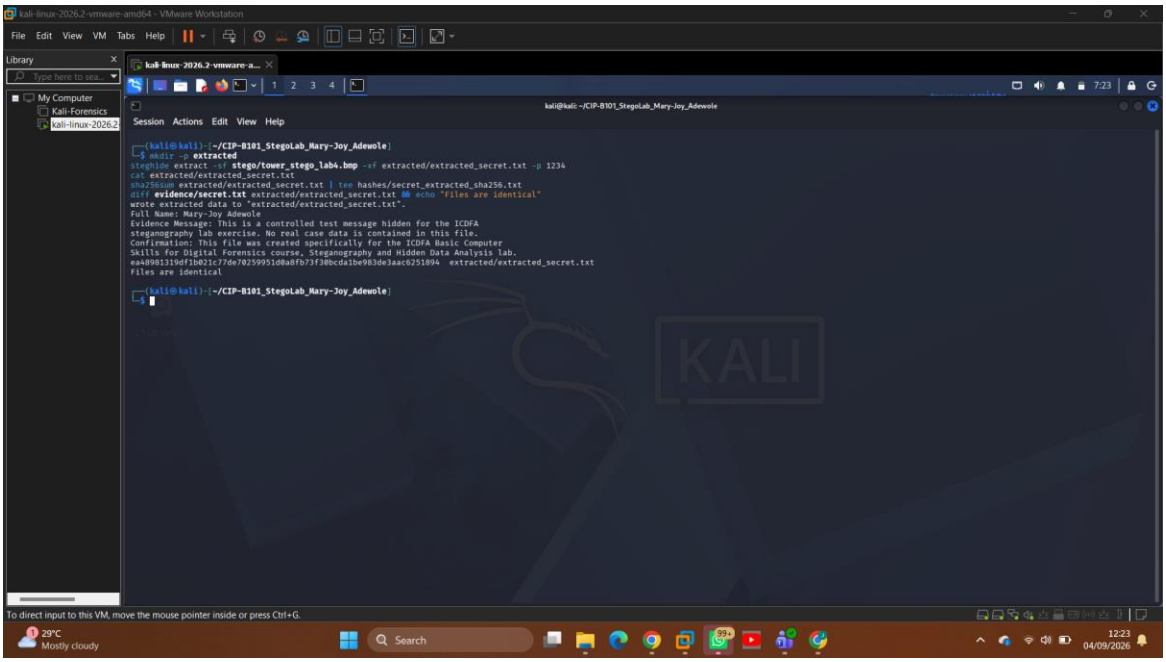


Figure 7: steghide extract recovering the hidden file; diff confirming "Files are identical"; SHA-256 of the extracted file recorded.

6. Part E - Original vs. Stego Image Comparison (Hashes and xxd)

The original carrier and the resulting stego image were compared directly: by file size and hash, and by raw byte content using xxd, to observe the effect of LSB embedding at the byte level. A strings search for the hidden content was also attempted directly against the stego image.

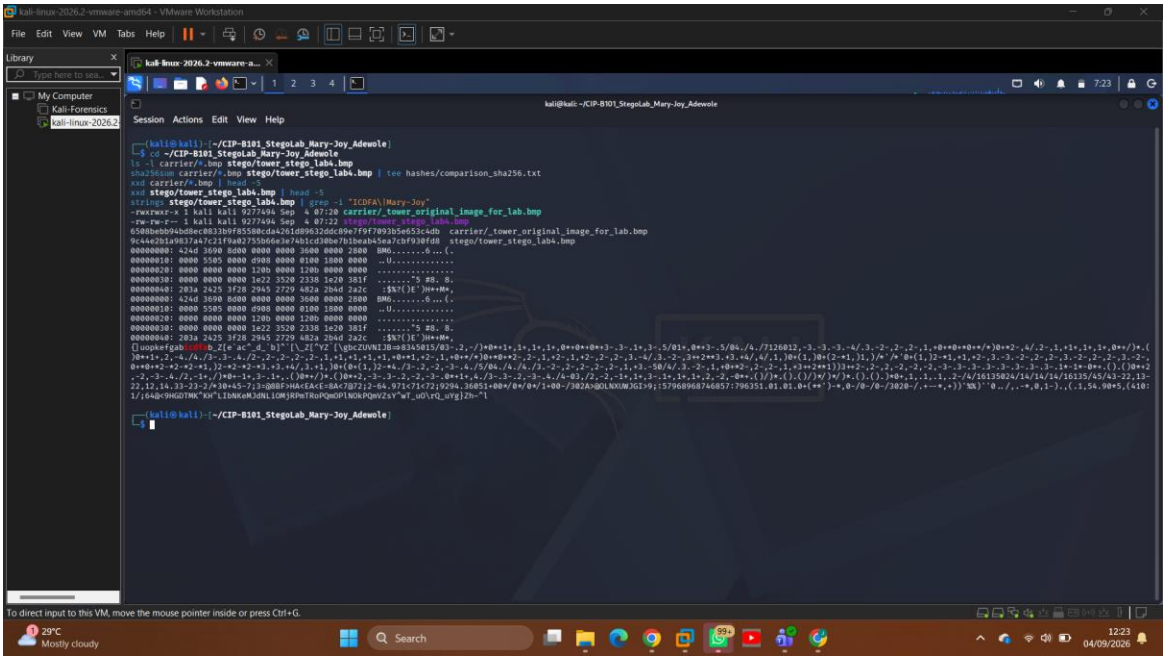


Figure 8: sha256sum confirming the carrier and stego image hashes differ; xxd showing the first five lines of both files are byte-identical (BMP header untouched); strings search for the embedded content.

Both files are 9,277,494 bytes and share an identical BMP header in the first bytes shown by xxd, confirming that Steghide's LSB substitution modifies only pixel data deep within the file rather than the file's structural header - exactly the behaviour expected of least-significant-bit steganography, where only the lowest-order bit of selected pixel bytes is altered to encode hidden data, keeping the carrier visually and structurally indistinguishable from the original. The strings search for "ICDFA" or "Mary-Joy" did not return the plaintext secret content; only an unrelated, coincidental substring match within pixel noise was found. This is expected: Steghide encrypts the embedded payload (AES, keyed by the supplied passphrase) before performing LSB substitution, so the hidden text is not recoverable as plaintext by a simple strings scan even though it is physically present in the file - the correct passphrase is required to both locate and decrypt it.

7. Part F - StegSeek Passphrase Recovery

A custom wordlist was created containing five candidate passphrases, including the correct one, and used with StegSeek to attempt automated passphrase recovery against the stego image.

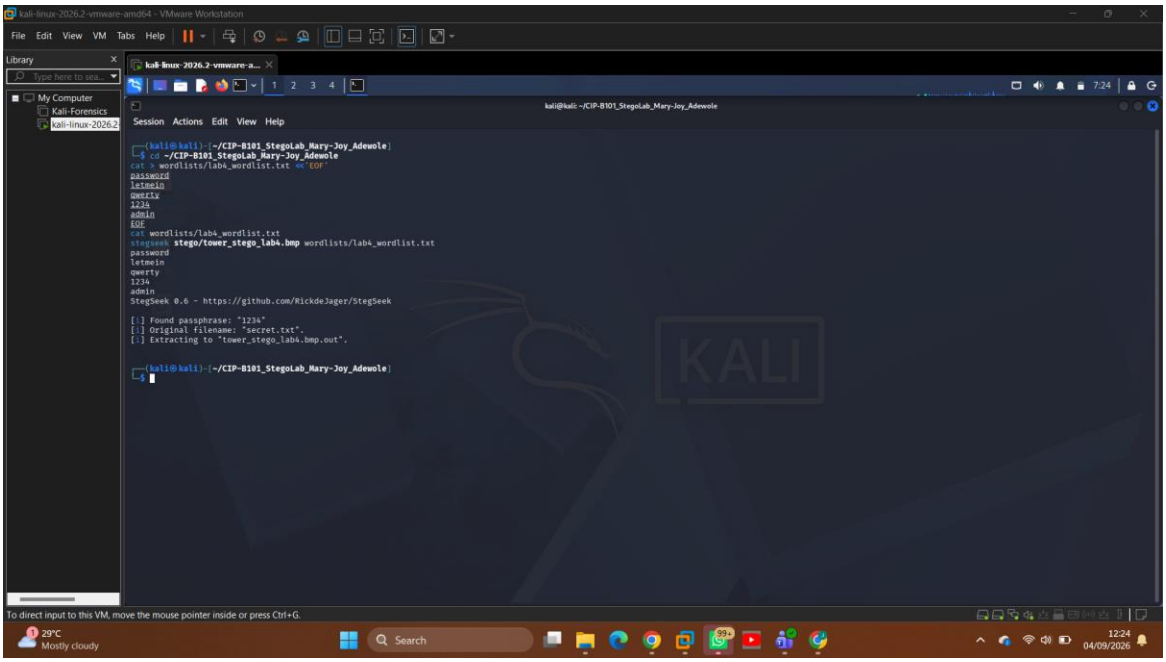


Figure 9: Custom wordlist (lab4_wordlist.txt) created and displayed; StegSeek successfully recovering the passphrase "1234" and the original filename "secret.txt".

Guided Exercise Summary:

Secret file	evidence/secret.txt
SHA-256 (original secret)	ea48981319df1b021c77de70259951d0a8fb73f30bcda1be983de3aac6251894
Passphrase	1234
Stego output	stego/tower_stego_lab4.bmp
SHA-256 (stego image)	9c44e2b1a9837a47c21f9a02755b66e3e74b1cd30be7b1beab45ea7cbf930fd8
Extracted file	extracted/extracted_secret.txt
SHA-256 (extracted)	ea48981319df1b021c77de70259951d0a8fb73f30bcda1be983de3aac6251894 -> MATCHES original
diff result	"Files are identical"
StegSeek result	Passphrase "1234" found via custom wordlist; original filename "secret.txt" recovered

8. Part G - Independent Steganography Exercise

A second, self-authored steganography exercise was performed independently of the guided scenario: a new secret file, a new passphrase, and a new wordlist were created, and the full embed-extract-verify-recover workflow was repeated in full using the same carrier image.

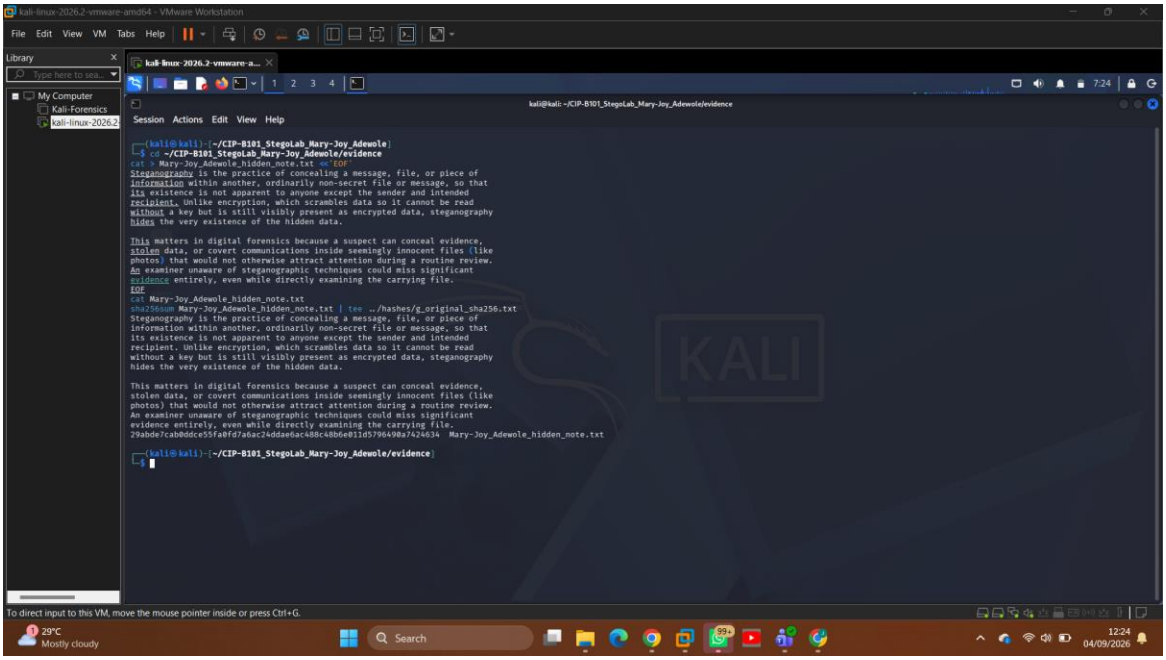


Figure 10: A second, independently authored secret file (Mary-Joy_Adewole_hidden_note.txt) created, explaining the concept and forensic relevance of steganography; content displayed and hashed.

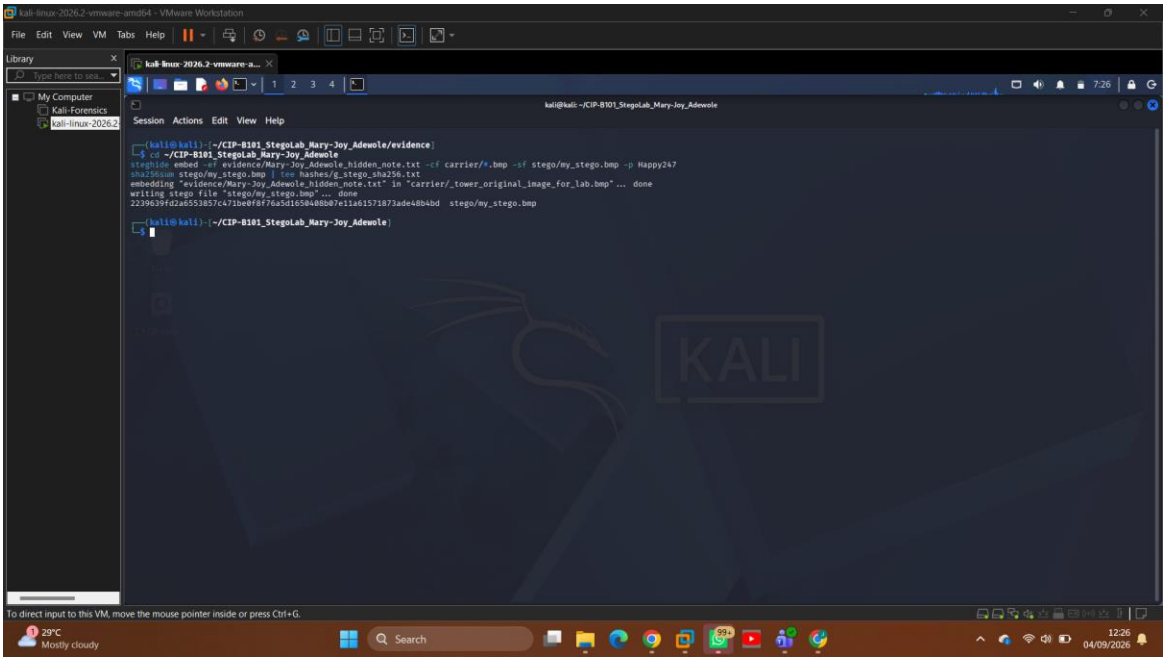


Figure 11: Independent steghide embed using passphrase "Happy247", producing my_stego.bmp; hash recorded.

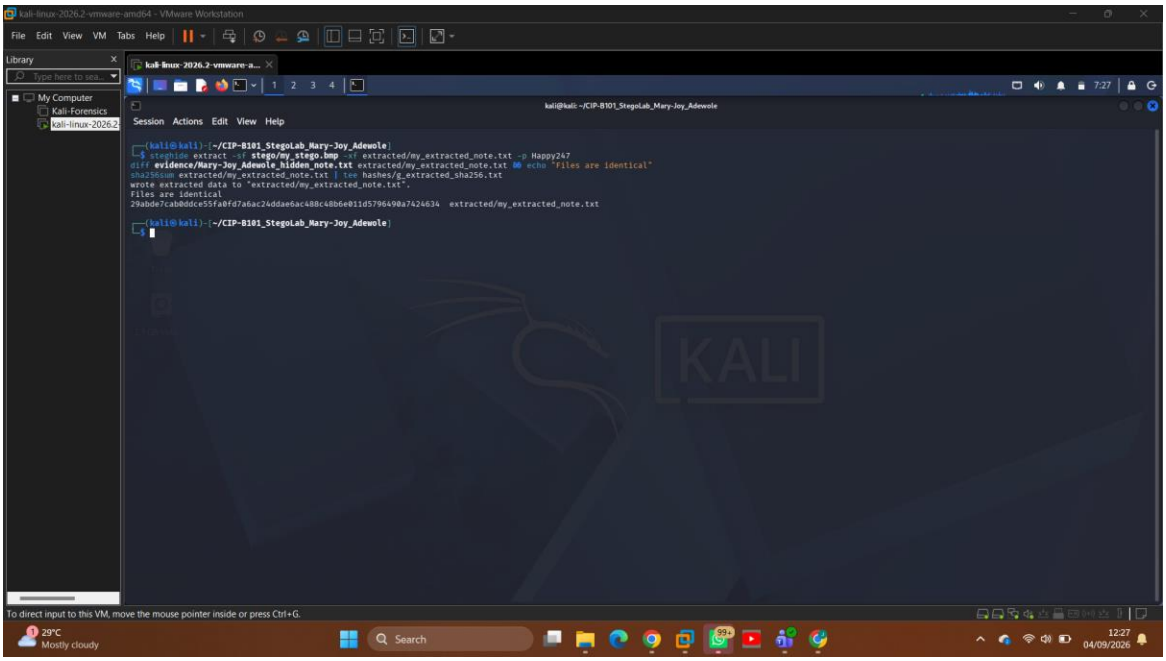


Figure 12: Independent extraction confirming "Files are identical" via diff, with a matching SHA-256 hash against the original secret file.

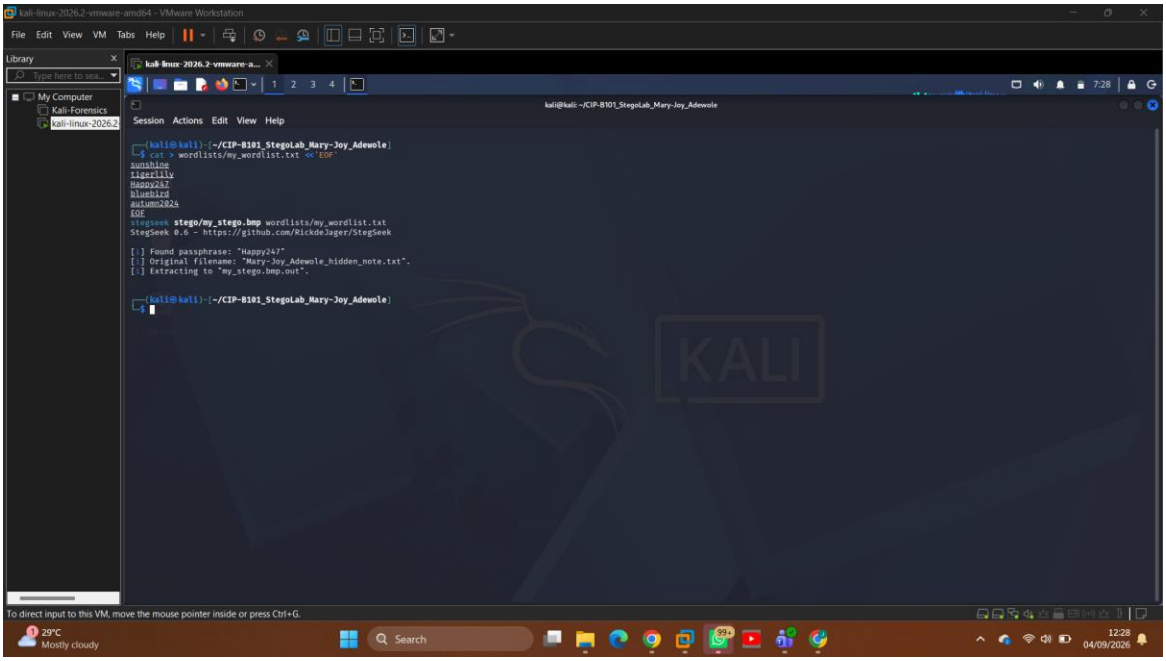


Figure 13: A second custom wordlist (my_wordlist.txt) created; StegSeek successfully recovering the passphrase "Happy247" and the original filename.

Independent Exercise Summary:

Secret file	evidence/Mary-Joy_Adewole_hidden_note.txt
SHA-256 (original secret)	29abde7cab0ddce55fa0fd7a6ac24ddae6ac488c48b6e011d5796490a7424634
Passphrase	Happy247
Stego output	stego/my_stego.bmp

SHA-256 (stego image)	2239639fd2a6553857c471be0f8f76a5d1650408b07e11a61571873ade48b4bd
Extracted file	extracted/my_extracted_note.txt
SHA-256 (extracted)	29abde7cab0ddce55fa0fd7a6ac24ddae6ac488c48b6e011d5796490a7424634 -> MATCHES original
diff result	"Files are identical"
StegSeek result	Passphrase "Happy247" found via custom wordlist; original filename "Mary-Joy_Adewole_hidden_note.txt" recovered

Wordlist Comparison:

Guided Exercise Wordlist (lab4_wordlist.txt)	Independent Exercise Wordlist (my_wordlist.txt)
password, letmein, qwerty, 1234, admin	sunshine, tigerlily, Happy247, bluebird, autumn2024

9. The LSB Substitution Concept

Least-significant-bit (LSB) substitution is the underlying technique Steghide uses to hide data inside the carrier image. Each pixel in a bitmap image is stored as a set of colour-channel byte values; the least significant bit of a byte has the smallest possible effect on the value it represents, so altering it changes the stored colour by at most one unit out of 256 - a change far too small to be perceptible to the human eye. By replacing the least significant bit of many pixel bytes throughout the image with the bits of a secret message (after encrypting that message with the supplied passphrase), the hidden data becomes embedded directly in the image's pixel data while the image continues to display and function normally. This explains two observations made directly in this practical: the carrier and stego images remained identical in file size and in their structural BMP header (Part E), because LSB substitution only touches pixel data, never the file's format metadata; and the hidden content could not be found by a plaintext strings search (Part E), because the message is encrypted before its bits are distributed across the image, rather than being stored as a contiguous readable block.

10. Evidence Integrity Summary

Every stage of both steganography exercises was independently verified. In each exercise, the extracted file's SHA-256 hash matched the original secret file's hash exactly, and diff confirmed the two files were byte-for-byte identical - together demonstrating that Steghide's embed/extract cycle introduced no data loss or corruption. StegSeek independently recovered the correct passphrase and original filename in both exercises purely from the stego image and a wordlist, without being told the passphrase directly, corroborating that the embedding had genuinely taken place as described. The carrier and stego image hashes differed as expected (confirming embedding altered the file), while the BMP header bytes remained identical (confirming the alteration was confined to pixel data, consistent with LSB substitution).

11. Artefacts

Artefact	Location	Size	Verification
_tower_original_image_for_lab.bmp	carrier/	9,277,494 bytes	Hashed
secret.txt	evidence/	text	Hashed
tower_stego_lab4.bmp	stego/	9,277,494 bytes	Hashed

extracted_secret.txt	extracted/	text	Hash + diff match
lab4_wordlist.txt	wordlists/	5 entries	StegSeek recovered passphrase
Mary-Joy_Adewole_hidden_note.txt	evidence/	text	Hashed
my_stego.bmp	stego/	9,277,494 bytes	Hashed
my_extracted_note.txt	extracted/	text	Hash + diff match
my_wordlist.txt	wordlists/	5 entries	StegSeek recovered passphrase

12. Challenges

No significant procedural issues were encountered in this practical. StegSeek's installation required an explicit apt upgrade rather than being present at its latest version by default, which was identified with which before proceeding and resolved with a single upgrade command.

13. Analysis and Observations

Running the full workflow twice - once against a guided scenario and once with independently authored material - produced consistent results across both: SHA-256 hash matches on extraction, identical diff outcomes, and successful StegSeek recovery, despite using different secret content, different passphrases, and different wordlists each time. This repeatability supports the conclusion that the observed behaviour is a property of the technique itself rather than a one-off result specific to the first exercise's inputs. It is also worth noting that both exercises used a passphrase that was deliberately included in the custom wordlist; StegSeek's recovery in this practical demonstrates that a weak or guessable passphrase drawn from a small candidate list is trivially recoverable, which is itself a relevant forensic and security finding: the strength of Steghide's encryption is only as good as the unpredictability of the passphrase chosen, independent of the LSB embedding technique's own robustness against visual or structural detection.

14. Limitations

This practical demonstrates detection and recovery only where the correct (or a guessable) passphrase is available via wordlist attack; a stego image embedded with a strong, non-dictionary passphrase would not be recoverable by StegSeek within a practical timeframe, and this practical did not test that scenario.

The xxd-based comparison in Part E inspected only the first five lines (BMP header) of each file; a full statistical steganalysis (e.g. chi-square attack on pixel LSB distribution) to detect the presence of hidden data without knowing a stego image was expected in advance was outside the scope of this practical.

15. Conclusion

This practical demonstrated the complete steganography lifecycle - embedding, extraction, integrity verification, and passphrase recovery - using Steghide and StegSeek against a BMP carrier image, performed twice to confirm consistent, repeatable results with entirely independent secret content and passphrases. Hash comparisons and diff confirmed lossless embedding and extraction in both cases, xxd comparison confirmed the LSB substitution technique's defining property of leaving file structure untouched while altering only pixel data, and StegSeek's

successful passphrase recovery in both exercises confirmed that a wordlist-based attack against a weak passphrase is practically effective. The original carrier image was preserved unmodified throughout, with all embedding performed into separate output files.